

C++ 프로그래밍 과제 3

표준입출력과 파일 디스크립터 분석

과제 목적

이 과제의 목적은 **C/C++ 프로그램의 표준입출력 구조와 운영체제의 파일 디스크립터(File Descriptor) 개념을 이해하는 것**이다.
본 과제를 통해 다음을 이해하는 것을 목표로 한다.

- 표준입출력 (`stdin` , `stdout` , `stderr`)
- POSIX 계열 운영체제의 파일 디스크립터(File Descriptor)
- Shell 리다이렉션
- 파이프(pipe)
- C++ 표준입출력과 OS IO 시스템의 관계

제출 형식

보고서(PDF) 형식으로 제출한다.

- 보고서 **첫 페이지**에는 보고서 **제목, 학과, 학번, 이름**을 반드시 기재한다.

보고서는 다음과 같은 구성으로 작성한다.

- 이론 설명
- 실험 과정
- 실행 결과
- 분석 및 결론

작성한 코드는 **보고서의 부록(Appendix)에 첨부**한다.

1. 이론 조사

다음 질문에 답하시오.

(1) 파일 디스크립터란 무엇인가

다음을 설명하시오.

- File Descriptor 정의
- 왜 정수(integer)로 표현되는가
- POSIX 계열 운영체제(Unix, Linux, MacOS) 내부에서 파일 디스크립터가 어떤 역할을 하는가?
- Windows 운영체제에서는 파일 디스크립터를 어떻게 다루는가?

(2) 표준입출력의 파일 디스크립터

POSIX 시스템에서 다음 값이 의미하는 것을 설명하시오.

이름	File Descriptor
<code>stdin</code>	0
<code>stdout</code>	1
<code>stderr</code>	2

다음을 설명하시오.

- 프로그램이 시작될 때 왜 이미 열려 있는가
- 누가 생성하는가 (OS / Shell)

(3) C/C++ IO와 파일 디스크립터

다음 관계를 설명하시오.

C++	C	OS
<code>std::cin</code>	<code>stdin</code>	fd 0
<code>std::cout</code>	<code>stdout</code>	fd 1

C++	C	OS
std::cerr	stderr	fd 2

다음을 설명하시오.

- C++ `iostream`과 OS IO 시스템의 관계
- `printf` / `cout` 의 내부 구조 차이

2. 표준입출력 프로그램 작성

다음 프로그램을 작성하시오.

```
#include <iostream>
#include <string>

int main() {

    std::string line;

    while (std::getline(std::cin, line)) {
        std::cout << line << std::endl;
    }

}
```

이 프로그램은 `stdin` 에서 입력을 읽고 `stdout` 으로 출력한다.

3. stdout / stderr 리다이렉션 실험

다음 프로그램을 작성하고, 실행파일명이 `test` 가 되도록 빌드하시오.

```
#include <iostream>

int main() {

    std::cout << "This is stdout message" << std::endl;
    std::cerr << "This is stderr message" << std::endl;

}
```

실험 1. 기본 실행

```
./test
```

출력 결과를 확인하고 설명하시오.

실험 2. stdout 리다이렉션

```
./test > out.txt
```

다음을 설명하시오.

- `stdout` 은 어디로 연결되는가
- `stderr` 는 어디로 출력되는가

실험 3. stderr 리다이렉션

```
./test 2> err.txt
```

다음을 설명하시오.

- `2>` 의 의미
- 숫자 `2` 의 의미

4. stdout 과 stderr 분리

```
./test > out.txt 2> err.txt
```

각 파일의 내용을 확인하고 설명하시오.

5. stdout 과 stderr 를 하나로 합치기

```
./test > all.txt 2>&1
```

다음을 설명하시오.

- 2>&1 의 의미
- 파일 디스크립터 관점에서 설명

6. stdin 리다이렉션 실험

다음 프로그램을 작성하고, 실행파일명이 read_input 이 되도록 빌드하시오.

```
#include <iostream>
#include <string>

int main() {

    std::string line;

    std::cout << "Reading from STDIN..." << std::endl;

    while (std::getline(std::cin, line)) {
        std::cout << "Input : " << line << std::endl;
    }

}
```

실험 1. 키보드 입력

```
./read_input
```

입력 예

```
apple
banana
orange
```

결과를 기록하시오.

실험 2. 파일로 stdin 대체

다음 파일을 생성한다.

```
input.txt
```

내용

```
apple
banana
orange
```

실행

```
./read_input < input.txt
```

다음을 설명하시오.

- 프로그램 코드를 수정하지 않았는데 파일을 읽는 이유
- Shell이 어떤 파일 디스크립터를 변경했는가

7. 파이프 실험

다음 명령을 실행하시오.

```
cat input.txt | ./read_input # Unix/Linux/macOS의 경우
type input.txt | ./read_input # Windows의 경우
```

또는

```
echo "hello world" | ./read_input
```

다음을 설명하시오.

1. pipe의 동작 원리
2. 프로그램의 `stdin` 이 무엇과 연결되는가
3. 파일 디스크립터 관점에서 설명

결론

보고서 마지막에 다음 질문에 답하시오.

1. 프로그램은 파일을 읽는 것이 아니라 무엇을 읽는가?
2. `stdin` / `stdout` / `stderr` 는 무엇인가?